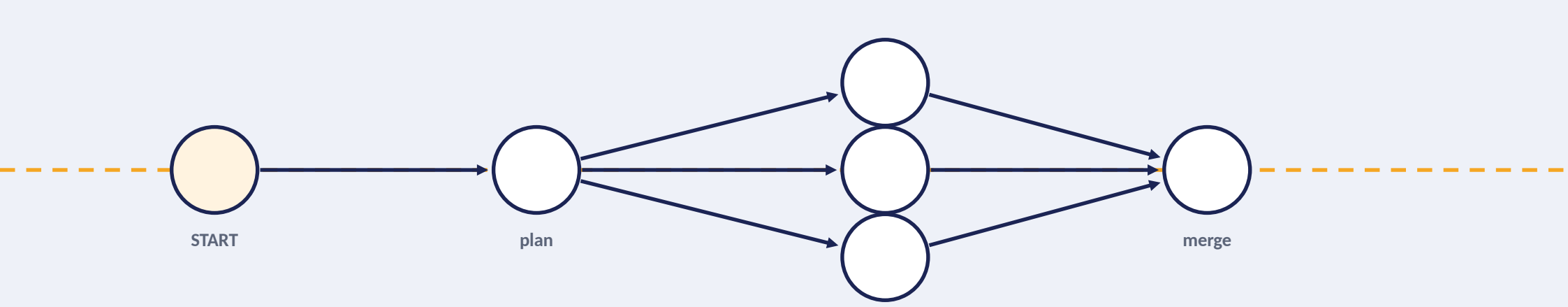
LANGGRAPH · CHAPTER 5 · FLASHCARDS

Workflow Patterns

Five shapes, wired by you

12 code cards on structured output, RAG and the five workflow shapes



A form, not an essay

with_structured_output returns a validated Pydantic object, not a paragraph to parse

- Code can branch on a typed field; ge/le limits are checked by Pydantic, not merely requested.
- Only constrained fields are enforced: a plain str like sentiment accepts "meh"; a Literal locks it.

PYTHON

```python
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI

class ProductReview(BaseModel):
    """Structured product review analysis."""
    product_name: str = Field(description="Name of the product")
    sentiment: str    = Field(description="positive, negative, or neutral")
    rating: int       = Field(description="Rating from 1-5", ge=1, le=5)
    pros: list[str]   = Field(description="List of positive aspects")
    cons: list[str]   = Field(description="List of negative aspects")
    summary: str      = Field(description="Brief summary of the review")

llm = ChatOpenAI(model="gpt-4o", temperature=0)
structured_llm = llm.with_structured_output(ProductReview)   # bind the schema

def analyze_review(state) -> dict:      # a node stores a validated object
    return {"analysis": structured_llm.invoke(state["review_text"])}

# Pydantic checks what the model fills in:
#   rating=7          -> ValidationError: Input should be less than or equal to 5
#   sentiment="meh"   -> accepted: a plain str is only described, not enforced
```

A fixed sequence, with a gate

A chain is focused calls in a fixed order, each writing one field the next one reads

- Each field is written once by one node, so no reducer; every stage stays in state for auditing.
- A gate is a conditional edge mid-chain: a good draft ends early, a weak one gets more stages.

PYTHON

```python
# The four-stage chain (01_prompt_chaining.py)
builder.add_edge(START, "generate_draft")
builder.add_edge("generate_draft", "fact_check")
builder.add_edge("fact_check", "improve_content")
builder.add_edge("improve_content", "format_output")
builder.add_edge("format_output", END)

# A quality gate after the first call (01_prompt_chaining_quality_gate.py)
MAX_WORDS = 50

def meets_brief(state: State) -> str:
    """The gate: a router, so it only reads state. Two checks code can do exactly."""
    words = len(state["draft"].split())
    has_price = state["price"] in state["draft"]
    return "pass" if words <= MAX_WORDS and has_price else "fail"

builder.add_conditional_edges("draft_description", meets_brief,
                              {"pass": END, "fail": "tighten_description"})
builder.add_edge("tighten_description", "add_call_to_action")
builder.add_edge("add_call_to_action", END)
```

Build the index once

Indexing is setup code: load, chunk, embed and store once, before the graph ever runs

- Each chunk becomes an embedding, a vector of its meaning; FAISS finds the nearest ones fast.
- rag_index.py sits at the repository root because Chapters 7 and 13 reuse the same index.

PYTHON

```python
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

def build_vectorstore(docs_dir) -> FAISS:
    """Load, split, and embed every .txt file in docs_dir into a FAISS index."""
    loader = DirectoryLoader(str(docs_dir), glob="**/*.txt", loader_cls=TextLoader,
                             loader_kwargs={"encoding": "utf-8"})
    raw_docs = loader.load()

    # Overlap keeps a sentence that straddles a boundary findable from either chunk.
    splitter = RecursiveCharacterTextSplitter(chunk_size=600, chunk_overlap=80)
    chunks = splitter.split_documents(raw_docs)

    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    return FAISS.from_documents(chunks, embeddings)     # embed every chunk, once

vectorstore = build_vectorstore(DOCS_DIR)   # not a node: run once at startup
```

RAG · PHASE 2

Retrieve, then generate

Local RAG is a two-node chain: look up passages, then answer only from them

- retrieve reaches the index by closure; building it inside the node would re-embed per question.
- "Only the retrieved context" keeps answers traceable; "say so" gives an honest exit on a miss.

PYTHON

```python
class RAGState(TypedDict):
    question: str
    retrieved: str      # written by retrieve
    answer: str         # written by generate

def build_graph(vectorstore):
    def retrieve(state: RAGState) -> dict:
        # A lookup, not an LLM call: embed the question, return the 3 nearest chunks
        docs = vectorstore.similarity_search(state["question"], k=3)
        return {"retrieved": "\n\n".join(f"[Chunk {i + 1}]\n{d.page_content}"
                                         for i, d in enumerate(docs))}

    def generate(state: RAGState) -> dict:
        prompt = ("Answer the question using only the retrieved context.\n"
                  "If the context is not enough, say so.\n\n"
                  f"Question: {state['question']}\n\nContext:\n{state['retrieved']}")
        return {"answer": llm.invoke(prompt).content}

    # builder: START -> retrieve -> generate -> END
    return builder.compile()

graph = build_graph(vectorstore)    # the graph closes over the prebuilt index
```

Classify once, then dispatch

One LLM call in a node writes a label; a plain-Python router branches on it

- A Literal schema means the model can return only one of three exact strings, never "mostly positive".
- The router's Literal return type lists every target, so the mapping dict is optional.

PYTHON

```python
class FeedbackCategory(BaseModel):
    category: Literal["positive", "negative", "neutral"] = Field(
        description="The category of the customer feedback")
structured_llm = llm.with_structured_output(FeedbackCategory)

def classify_feedback(state: State) -> dict:     # NODE: the only LLM call
    result = structured_llm.invoke([HumanMessage(
        content=f"Classify this customer feedback: {state['feedback']}")])
    return {"category": result.category}

def route(state: State) -> Literal["positive_node", "negative_node", "neutral_node"]:
    if state["category"] == "positive":           # ROUTER: no model, just reads
        return "positive_node"
    elif state["category"] == "negative":
        return "negative_node"
    else:
        return "neutral_node"

builder.add_conditional_edges("classify_feedback", route)  # 3 edges from the hint
for name in ("positive_node", "negative_node", "neutral_node"):
    builder.add_edge(name, END)                  # every branch needs its own exit
route({"category": "negative"})   # 'negative_node': testable with no model
```

Fan out, fan in

Several edges out of START run branches in one step; several edges in make a node wait

- No reducer needed: total, difference and product are each written by exactly one branch.
- Order inside a step is not guaranteed; that combine runs last, after all three, is.

PYTHON

```python
def add_node(state: MathState) -> dict:
    return {"total": state["x"] + state["y"]}      # each branch owns one field

# subtract_node -> "difference", multiply_node -> "product": same shape

def combine_node(state: MathState) -> dict:
    return {"summary": f"sum={state['total']}, difference={state['difference']}, "
                       f"product={state['product']}"}

# graph = StateGraph(MathState), then add_node for add, subtract, multiply, combine
for branch in ("add", "subtract", "multiply"):
    graph.add_edge(START, branch)       # fan-out: all three start together
    graph.add_edge(branch, "combine")   # fan-in: combine waits for all three
graph.add_edge("combine", END)
app = graph.compile()

for update in app.stream({"x": 6, "y": 3}, stream_mode="updates"):
    print(update)
# {'add': {'total': 9}}               <- one step: the three branches,
# {'multiply': {'product': 18}}       <- in no promised order
# {'subtract': {'difference': 3}}
# {'combine': {'summary': 'sum=9, difference=3, product=18'}}
```

Two writers, one field

Parallel writes to one plain field raise InvalidUpdateError; a reducer merges them

- Return a one-item list: the add reducer appends it; returning the whole list adds it all again.
- The reducer applies writes in a fixed order (by node name), not in the order branches finish.

PYTHON

```python
class SharedNoReducer(TypedDict):
    x: float
    y: float
    results: list[str]                    # no reducer: one value per step

class SharedWithReducer(TypedDict):
    x: float
    y: float
    results: Annotated[list[str], add]    # reducer: concatenate every write

def add_line(state) -> dict:              # subtract_line, multiply_line: same shape
    return {"results": [f"{state['x']} + {state['y']} = {state['x'] + state['y']}"]}

# build_shared_graph(schema): START -> add | subtract | multiply -> END
inputs = {"x": 6, "y": 3, "results": []}

build_shared_graph(SharedNoReducer).invoke(inputs)
# InvalidUpdateError: At key 'results': Can receive only one value per step. ...

build_shared_graph(SharedWithReducer).invoke(inputs)["results"]
# ['6 + 3 = 9', '6 x 3 = 18', '6 - 3 = 3']   same order on every run
```

One worker per planned item

A dispatch function returns a list of Send objects: N workers, with N decided at runtime

- Each Send carries a private input, so a worker sees only its own word, not the whole state.
- All workers write lengths at once, so it needs a reducer; combine waits for every worker.

PYTHON

```python
class OverallState(TypedDict):
    sentence: str
    words: list[str]                        # written once, by plan
    lengths: Annotated[list[str], add]      # every worker appends one line
    summary: str                            # written once, by combine

class WorkerState(TypedDict):
    word: str                               # a worker sees only its own word

def dispatch(state: OverallState) -> list[Send]:
    return [Send("measure", {"word": word}) for word in state["words"]]

def measure(state: WorkerState) -> dict:
    return {"lengths": [f"{state['word']}={len(state['word'])}"]}

builder.add_edge(START, "plan")                          # plan splits the sentence
builder.add_conditional_edges("plan", dispatch, ["measure"])   # Sends, not a name
builder.add_edge("measure", "combine")        # then combine -> END
# 'graphs pass batons'                      -> workers: 3
# 'one worker per word decided at runtime'  -> workers: 7  (same compiled graph)
```

Try, judge, revise

A reviewer returns a typed verdict plus a critique the writer uses on its next pass

- Cheap code checks (length, regex, tests) run first; the model judges only what needs judgment.
- The writer gets its last draft and the critique, so it fixes what was flagged instead of restarting.

PYTHON

```python
class Review(BaseModel):
    verdict: Literal["approve", "revise"] = Field(
        description="approve only if the reply follows every rule in the policy")
    critique: str = Field(description="If revise: the specific changes to make, "
                                      "as instructions. If approve: empty.")

reviewer = llm.with_structured_output(Review)
def review_reply(state: State) -> dict:
    rounds = state.get("rounds", 0) + 1
    words = len(state["draft"].split())
    if words > WORD_LIMIT:      # code counts words exactly, and for free
        verdict = "revise"
        critique = f"The reply is {words} words. Cut it to under {WORD_LIMIT}."
    else:
        review = reviewer.invoke(f"...policy:\n{REPLY_POLICY}\n...{state['draft']}")
        verdict, critique = review.verdict, review.critique
    return {"verdict": verdict, "critique": critique, "rounds": rounds}

builder.add_conditional_edges("review_reply", route_after_review,
                              {"approve": END, "revise": "draft_reply"})
```

A signal and a guarantee

The verdict says "good enough"; a round counter says "stop regardless". Ship both

- Out of rounds goes to escalate, not END: an unapproved draft is a case for a person.
- Without the cap only the recursion limit stops it: 10,007 steps, about 5,000 paid rounds.

PYTHON

```python
MAX_ROUNDS = 3

def route_after_review(state: State) -> str:
    """Three exits. The verdict is a signal; the round count is the guarantee."""
    if state["verdict"] == "approve":
        return "approve"
    if state["rounds"] >= MAX_ROUNDS:
        return "out_of_rounds"
    return "revise"

def escalate(state: State) -> dict:
    """Out of rounds: hand the last draft and the open critique to a person."""
    return {"status": "escalated"}

builder.add_conditional_edges("review_reply", route_after_review,
                              {"approve": "approve", "revise": "draft_reply",
                               "out_of_rounds": "escalate"})

# A real run with a strict 40-word policy:
#   round 1: revise (149 words)    round 2: revise (promised a refund)
#   round 3: revise (51 words)     status: escalated after 3 round(s)
```

The task names its shape

Use the most constrained shape that fits; reach for an agent only when paths can't be listed

- Patterns compose: a router can send one request to a RAG chain and another to a fan-out.
- Most tasks that feel like they need an agent are a workflow with two or three branches.

TEXT

```
TASK                                PATTERN               KEY MECHANISM
code must act on model output       structured output     with_structured_output
known, ordered sequence             prompt chaining       one field per node
must use your own documents         retrieve -> generate  index + grounded prompt
handling varies by category         routing               Literal label + router
N independent subtasks, N fixed     parallelization       fan-out, own fields
N subtasks, N known at runtime      orchestrator-workers  Send + a reducer
draft, judge, redo until good       evaluator-optimizer   loop + critique + cap
paths cannot be listed ahead        an agent: Chapter 7   model picks each step

Pick the first row that fits. Patterns nest:
  route -> [ RAG chain | fan-out -> workers -> review loop ]
```

Three quiet bugs

A duplicated list, a fan-in that never runs, and a loop that never exits

- With an add reducer, return only your delta; with Send, decide what an empty plan should do.
- Test loops with a small recursion_limit; the default lets a broken exit run thousands of rounds.

PYTHON

```python
# 1. Returning the whole list into an add reducer re-adds the old items
def node_a(state):
    return {"results": state["results"] + ["a"]}     # wrong
# start ['x'], then a, then b ->  ['x', 'x', 'a', 'x', 'x', 'a', 'b']
def node_a(state):
    return {"results": ["a"]}                         # right: only the delta

# 2. A plan with zero items: zero Sends, so combine never runs (04a_send_minimal.py)
graph.invoke({"sentence": ""})
# {'sentence': '', 'words': [], 'lengths': []}   <- no 'summary', no error

# 3. A misspelled verdict: the loop can never take the approve exit
def route_after_review(state):
    return "approve" if state["verdict"] == "approved" else "revise"   # typo
graph.invoke(inputs, config={"recursion_limit": 12})   # cap the experiment
# GraphRecursionError after 6 rounds, though round 3 was approved

# Checked on langgraph 1.2.6 with plain-Python nodes
```